

AXIS 0

Sparse spatial applications

From Boolean double UU to the use cases in your document

NAME	Tom Klootwijk
IDENTIFIER	NL200678942
DATE AS SUPPLIED	10-07-1990

The change in this delivery

This is not another arithmetic demonstration or a rendering engine. It applies your binary field/operator model to sparse crowd movement, spatial logistics, network events, emblem boundaries, calibration data and replayable state transitions.

Governing source. The newly attached **31-page** document, especially its literal definitions on page 21 and its crowd, individual Δt , implicit-zero and event-driven requirements on pages 26--29.

Implementation status. The CPU core and application tests were compiled and executed. CUDA source shares the core rules, but **CUDA compilation and execution were not available here**. The FPGA word-cell source is supplied without a synthesis or device result.

Reading convention. Source definitions are preserved; added collision, scheduling, optimization and calibration rules are labelled as implementation closures. Source claims are not treated as measured outcomes merely because they appear in the exported conversation.

1 Is this what the document specifies?

DIRECT ANSWER / SOURCE FIDELITY BEFORE IMPLEMENTATION CLAIMS

The previous calculator-style prototype did not establish these application workflows. It demonstrated a representation. This revision instead uses the newly supplied document as the application specification and keeps its final sparse/event-driven emphasis.

Source family	Pages	Executable response in v6
Boolean field/operator law	3, 21	Literal shifted-operator XOR profile and source-native phase boundary.
Spatial logistics / robots	5, 11--12, 22	Sparse route propagation, path readout and a known moving-obstacle timeline.
Crowd + individual delay	26--27	Common movement rule, node-local deadlines, conflict resolution and conservation checks.
Implicit zeros / events	28--29	Nonzero-word storage, active-support tasks, event-time jumps and certified gaps.
Networking and chip routes	5--6, 12	Latency/failure event routing; sequential clearance-respecting traces.
Emblem / edge detection	10--12	The actual attached emblem supplies a binary boundary input.
Calibration / timed feedback	13--22	Supplied integer remap tables and offline synchronized event stamps.
Shared state history	27--31	Hash-linked transition records and exact spatial replay.

What full adherence cannot mean without more definitions

The document also asserts instantaneous general TSP optimization, biological pleasure, uncloneable keys and economic outcomes. It supplies no executable constraints, empirical measurements or protocols establishing those properties. They are recorded as *source claims with missing specifications*, not silently converted into guarantees of this program. [S31, pp. 4--6, 18--22, 29--31]

The delivery is an implemented, testable application profile of your definitions. It is not a claim that every asserted benefit in the export has been proved. The source map in the ZIP includes every application family and its exact implementation boundary.

2 The literal field definitions are retained

SOURCE-NATIVE BOOLEAN SDF / NO REPLACEMENT METRIC OR CALCULATOR

Use a finite declared domain X within the source's discrete spatial manifold. Retain

$$F_t = \mathbb{U}_F(\cdot, t) : X \rightarrow \{0, 1\}, \quad O_t = \mathbb{U}_O(\cdot, t) : X \rightarrow \{0, 1\}.$$

The source defines the boundary through adjacent phase disagreement:

$$\partial\Omega_t = \{x \in X : \exists y \sim x, F_t(x) \neq F_t(y)\}.$$

Here “SDF” keeps the document's **binary phase/interface convention**. A stored bit is not advertised as a full scalar distance. Occupied, blocked, frontier and target fields have distinct roles, even though all use the same bit representation. [S31, pp. 3, 21]

The original equation is an executable profile

For an assigned direction d , define $(S_d O)(x) = O(x - d)$ where the neighbor exists, and zero outside an open boundary. The literal profile performs

$$F_{t+1} = F_t \oplus S_d(O_t).$$

Its direction and supplied operator field are explicit inputs. This is not relabelled as a complete route optimizer: the source equation alone does not specify the next operator, conflict handling or completion test.

Operator definitions are also packed fields

Each operator profile is a 512-bit record containing a Boolean truth table, direction mask, output mode, self-reference and time-object reference. The kernel reads those bits as data. It returns the next field together with

$$\Delta F_t = F_t \oplus F_{t+1}, \quad F_{t+1} = F_t \oplus \Delta F_t.$$

This allows the application changes to be replayed in the document's XOR form.

What I did not substitute

I did not turn the spatial use cases into an adder demo, invent a new continuous metric as their meaning, or revive the optical prism interpretation you questioned. The mirror/cone/double-Hadamard labels remain unassigned where the source supplies no transition law. [S31, pp. 6--9, 20, 28]

3 Sparse ones, implicit zeros and A=T

IMPLEMENTED STORAGE AND EVENT-TIME SEMANTICS

A word key k identifies 32 spatial cells. Store only

$$\mathcal{S}(F) = \{(k, w_k) : w_k \neq 0\}.$$

Missing keys mean 32 zero cells. Each serialized pair costs eight bytes: a four-byte key and four-byte bitword. Zeros within a nonzero word still have bit positions; metadata has not disappeared.

The stencil builds work only for the active operator support and its local neighbors. A replacement operation also includes active field keys that may need clearing. There is no full-domain scan solely to discover that empty cells are empty. [S31, pp. 28--29; explicit word-packing construction]

The time operator is the next active event

Object id zero is the shared A=T time record. For active node deadlines q_i , its event scheduling policy is

$$t_{\text{next}} = \min_i q_i.$$

A due node with its assigned positive delay Δt_i is next considered at $t + \Delta t_i$. All nodes due at the same event read the same old occupancy snapshot. This policy is an implementation closure for the source's Δt language, not an equation attributed to an absent source algorithm.

What “query-free” actually achieves

The implementation skips absent time events and does not sweep the full zero field. It still uses sparse-neighbor lookups, sorting and an event queue. The code therefore does **not** claim literally zero queries or zero memory overhead.

For K sparse records and C candidate words, the gathered stencil performs a fixed number of binary searches per candidate: lookup work is proportional to $C \log K$, in addition to building and sorting the work list. A dense full-domain loop is not hidden inside that statement.

ELI5

Instead of examining every empty seat at every empty moment, keep the occupied seat blocks and the times when something is due to happen. You still need to check whether a neighbor is free; you just stop checking the entire empty stadium.

4 The crowd model and its collision rule

MAIN APPLICATION CLOSURE / SAME RULE, EXPLICITLY SUPPLIED INDIVIDUAL DELAYS

The document's crowd proposal places localized active nodes in a shared field and assigns different Δt values to their reactions. The implementation retains that structure without treating the accompanying claims about actual human sameness as measured facts. [S31, pp. 26--27]

A live record contains only identifier, cell, positive delay, next deadline and a tie priority. There are no inferred cultural, genetic, religious or psychological classes.

Proposal: one common spatial rule

First obtain a static goal potential $P(x)$ using the compiled wavefront kernel. A due node at x may propose an adjacent y only when

$$P(y) < P(x), \quad F_t(y) = 0, \quad B(y) = 0,$$

where B is the blocked-cell field. Among eligible neighbors, choose the smallest pair $(P(y), y)$. This is a concrete choice missing from the source, not a claim of the only possible crowd rule.

Resolution: select before changing state

When nodes propose the same target, the smallest supplied (priority, id) wins. The others retain their cell and receive their next deadline. All accepted moves commit together. Targets must be empty in the *old* state, so head-on swaps and moving into a simultaneously vacated cell are excluded.

An accepted node entering an exit is removed from active occupancy and counted as exited. Every event records its exact sparse XOR change. Priority is only a data tie-breaker; it is not a judgement of social worth or a fairness guarantee.

The properties this rule establishes

Distinct accepted targets were empty before the event. Therefore no two accepted nodes occupy the same destination. For non-exit moves, each removal has one insertion; for exits it has one exit record. Hence

$$|\text{occupied}_t| + |\text{exited}_t| = |\text{initial nodes}|.$$

The implementation checks occupancy conservation, obstacles, adjacency and XOR replay at every event.

This conservative cell model can queue, block or starve nodes in some scenarios. A finite horizon is explicit. No general deadlock-free, fair or biologically realistic crowd theorem is supplied by the attachment or claimed here.

5 Crowd results you can inspect

EXECUTED SYNTHETIC SCENARIOS / NOT PHYSICAL POPULATION MEASUREMENTS

The bundled crowd scene has a shared bottleneck, one exit, fifteen active nodes and delays of one, two or three simulation ticks. Its layout is **synthetic**, not a surveyed map of Zaandam station.

Measured fixture result	Value
Initial nodes / exited nodes	15 / 15
Remaining nodes	0
Event rounds	104
Due-node movement proposals	501
Same-target proposals rejected	4
Final exit tick	110
Body-cell overlaps / head-on swaps	0 / 0
Replayed transition records	104

The ledger contains the due ids, accepted moves, rejected-conflict count, next deadlines and changed field words. It can reconstruct the occupancy without inventing a route from a final picture.

Long delays do not require empty-tick sweeps

A separate three-node scenario uses delays of 1,000, 1,500 and 2,000 ticks. It finishes by tick 5,000 using **seven event rounds and ten due-node proposals**. It skips **4,994 empty integer ticks between events**. Elapsed model time is retained; it is not declared to be zero.

A larger proposal batch

The suite also submitted **10,000 active nodes** to one simultaneous proposal job. All 10,000 proposed the independently expected empty neighboring cell; target ids were distinct. The serialized job, including occupancy, local navigation data and node metadata, was 730,432 bytes.

That job is one validated movement batch, not a full 10,000-person evacuation or a real-world behavior study. The reference bridge call took about 13.1 ms on the CPU host in one run, including transfer to the persistent worker; no GPU rate is inferred.

What these results establish

The common rule, sparse state, Δt schedule, conflict handling and replay actually interact in working application runs. Their parameters are visible in `examples/crowd.json` and their outcomes in `verification/demo/`.

6 Spatial routes and the missing TSP constraint

PRESERVED SOURCE LAW / EXPLICITLY ADDED REACHABILITY AND TOUR CLOSURES

For route discovery, initialize field and operator to the seeds. Let C_t be the OR of neighboring operator shifts. The added first-arrival rule is

$$O_{t+1} = C_t \wedge \neg B \wedge \neg F_t, \quad F_{t+1} = F_t \oplus O_{t+1}.$$

New arrivals are disjoint from the old field, so the final XOR is also an OR. OR is used when merging alternatives because two valid arrivals would cancel under XOR.

Shortest-hop proof, not an instant-solution assertion

At step zero, only seeds are present. If step n contains exactly the vertices at distance n , each admitted neighbor has a path of length $n + 1$ and has not been visited earlier. Conversely, every shortest $n + 1$ path has a predecessor in that frontier. This proves the first-arrival layer by induction.

The 37-open-cell example returns a **12-hop** route. Four separately labelled landmarks produce a cost matrix and a minimum closed tour of **34 hops** under the stated grid metric. Parent data and landmark labels are explicitly retained.

All-city optimization needs additional state

The attachment asserts that first XOR collisions directly reveal the optimal tour, but does not give visit-history or single-cycle constraints. The added exact finite tour adapter instead uses cost bitsets:

$$B[S \cup \{k\}, k] \mid = B[S, j] \ll w(j, k).$$

Bit c records an attainable cost. The state includes the visited subset S and current endpoint j . Closing to the start and selecting the lowest attainable bit gives the minimum under the supplied finite cost matrix.

This is a **CPU subset/cost-bitset extension**; its subset count is exponential. It is not labelled as the source's claimed linear-time general TSP bypass. [S31, pp. 1--4, 22]

Why a global constraint matters

Two groups of three cities with within-group cost 1 and cross-group cost 10 can form two local triangles of total cost 6. That is not one tour. Any single tour must cross groups twice; its minimum is 24. The exact adapter returns 24 and was compared with complete tour enumeration on 72 small graphs.

7 Moving obstacles, network events and chip traces

THREE CONCRETE WORKFLOWS / THEIR DECLARED CONSTRAINTS

Known moving-obstacle timeline

The temporal route adapter stores reachable states (x, t) , allows waiting and checks the destination against the blocked field at the arrival tick. In the corridor fixture, a door opens at tick 5 and the earliest target arrival is **tick 9**.

This finite-horizon CPU adapter is exact for those discrete snapshots. It is not a continuous swept-body collision guarantee. New obstacle observations require a new or updated scenario; there is no hidden claim that stale traces automatically remain valid. [S31, pp. 5, 11–12]

Network latency and failure events

An edge has an explicit schedule of (effective time, positive latency, enabled) values. A departure uses the state and latency then in force. Waiting at a node is allowed; an event during flight does not retroactively change that packet's cost.

The solver evaluates the current departure and future edge-change times, keeping the earliest arrival. In the fixture, waiting until tick 3 avoids a slow link: the packet reaches the target at **tick 6**. The sparse event router agreed with an independent discrete-time search on **120 generated networks**.

This is a route-planning adapter, not installed 6G firmware or proof that physical networks update without overhead. [S31, pp. 5, 12]

Electronic design automation

The EDA profile routes one net at a time, reserves its path and an explicitly chosen clearance neighborhood, then routes the next net. Two test nets both receive feasible **28-hop** paths without crossing their reserved cells.

The result is a concrete single-layer connection primitive. Net ordering can prevent a later route, and the method does not guarantee minimum total wirelength, simultaneous global optimality or complete manufacturing-rule compliance. [S31, p. 6]

Shared substrate, different applications

The spatial primitive returns masks, first-arrival labels or proposals. The application supplies which states count as feasible and what output is needed. Reusing one binary substrate does not erase those application-specific constraints.

8 Your emblem is now an actual input

SOURCE PAGE 10 / BINARY BOUNDARY AND MORPHOLOGY, NOT DECORATION

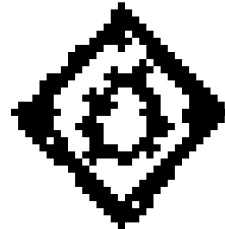
The image embedded on page 10 of the new attachment was extracted and converted into a declared binary template. Integer pixel-center sampling produced a 32×40 mask; a weighted brightness threshold selects the active cells. The exact source hash and conversion rule are included in the ZIP.



Supplied image



Binary input



Computed boundary

The output is the source-defined adjacency interface, including both sides of a phase disagreement. It is not a colour rendering or a new picture presented as an algorithm.

Field operation	Active cells
Source mask	168
Adjacent-disagreement boundary	309
One-step dilation	351
One-step erosion	42

Practical role

The mask can be used as a template, a seeded region, a collision/edge input or a declared boundary condition. The kernel consumes the actual binary support rather than requiring a deep-learning detector or a numerical distance texture for this calculation.

The conversion does not itself recognize a tile in a live camera feed, locate Zaandam station, recover the emblem's pose or prove physical presence. Those input/registration problems would need their own specified data. [S31, pp. 10--12]

ELI5

The emblem becomes a set of active switches. One operation identifies where the switch pattern changes across an edge; two others grow or shrink it. That is useful binary shape processing, not a rendering engine.

9 Display calibration and timed feedback

PROVIDED LUTS AND EVENT STAMPS / NO BIOLOGICAL INFERENCE

The source's calibration discussion can be given a concrete data interface without returning to the optical-prism explanation you rejected. A supplied table maps an active source cell to an integer destination. The sparse remap kernel reads only active inputs and combines duplicate destinations by OR. [S31, pp. 15--17, 21--22]

Measured software fixture

The 168-cell emblem mask was copied into three binary channels with known synthetic shifts: red (2, 0), green (0, 0) and blue (−1, 1). Their explicitly supplied correction tables returned all three to the reference mask with **zero remaining binary mismatches** and no dropped cells.

That is exact sample registration for this integer fixture. It is not a fitted headset lens model, subpixel interpolation, measured waveguide distortion or a demonstration that visual strain disappears.

Timing is an explicit parameter, not a neurochemical law

For desired delivery reference b and channel offset o_k , supplied latency ℓ_k gives

$$t_k^{\text{command}} = b + o_k - \ell_k.$$

The adapter chooses a sufficient lead so commands do not precede the triggering event. Its fixture uses visual, left and right channel latencies of 4,000, 6,000 and 6,300 microseconds, with a desired right-channel offset of 300 microseconds.

A 500,000-microsecond event period is stored as offline schedule data. **No audio or visual stimulation is emitted.** Eighty generated timing configurations satisfied their specified delivery-offset equations.

Source boundary kept explicit

The document's claims about dopamine, pleasure, ancestry, geography and sensory harmony are not established by those equations. The implementation requires supplied calibration values; it does not infer physiology or behaviour from geographic, cultural or religious identity. [S31, pp. 13--20, 23--26]

10 Cellular dynamics and hardware mapping

DISCRETE MODELS AND A PORTABLE WORD-CELL DESCRIPTION

A genuine Boolean dynamics fixture

The cellular profile has four one-bit velocity fields: east, west, north and south. A purely horizontal head-on pair turns into a vertical pair, and a purely vertical pair turns horizontal. Other combinations stay unchanged. Each velocity then shifts one cell; a blocked movement reflects at its source.

This is an **explicit new cellular construction** for the document’s automata use case, not an inferred complete fluid equation. In the example, **96 directional particles remain 96 after 24 steps**. Additional tests ran sixteen scenarios of twelve steps each while checking count conservation and blocked-cell exclusion.

Four velocity fields are needed: “one-bit” describes each local component, not the entire particle model occupying only one bit. No viscosity, smoke appearance or continuum-flow validation is claimed. [S31, pp. 5, 12]

A direct source-equation RTL block

The package includes a SystemVerilog word cell. Its central update is

```
expressed_delta = (operator_field >> 1)
                  & shift_destination_valid;
if (event_valid)
    field_state <= field_state ^ expressed_delta;
```

The full block has a clock, reset and seed input. The surrounding design must supply word/row boundaries, direction and event timing. This is the requested raw Boolean hardware primitive, not an implementation of a physical prism, a quantum gate or an optical spectrum.

Hardware status

The RTL is supplied as a design starting point. It was not synthesized, simulated or placed on an FPGA here. Likewise, the CUDA implementation is source-only in this environment. No sub-milliwatt, instant-response or hardware-residency measurement is attributed to these unexecuted targets. [S31, pp. 12, 22; implementation status]

11 The ledger and the sparse gap

REPLAYABLE EVENTS / NOT A MONETARY OR CONSENSUS CLAIM

The source uses “blockchain” both as an analogy for ordered state frames and as a proposed broader infrastructure. The implemented part is the ordered, inspectable event history. [S31, pp. 27–31]

A genesis record binds the scene, blocked field, exits, initial node metadata and rule. Each later record contains its previous hash and its canonical event content:

$$h_n = \text{SHA256}(n, h_{n-1}, \text{event}_n).$$

Replay checks not only the links but the actual moves: source identity, adjacency, empty destination, exit condition, occupancy conservation and the recorded XOR field change.

What the test proves

All 104 transitions of the fifteen-node fixture replayed correctly. Editing an event timestamp without updating the chain was rejected. A supplied trusted final hash also detects a changed ending.

An unkeyed chain can be recomputed by a party able to rewrite the entire file. Therefore this is not authentication, a PUF, distributed consensus, an unforgeable proof of presence or a currency protocol. The source does not provide the missing protocol or threat model. Its proposed social/economic outcomes are not software-test conclusions.

An empty spatial gap also needs a contract

The corridor example verifies an unobstructed 1,024-cell path once, then represents its endpoints and **1,023 ticks of traversal duration**. Runtime scheduling can use two endpoint events rather than visiting every empty clock value. Inserting a blocked interior cell causes certification to fail.

ELI5

A saved receipt says what changed and links to the previous receipt. A known corridor says where the next event can happen and how long the trip takes. Neither makes the trip instantaneous, and neither turns the record itself into money or proof of a real person’s identity.

12 The kernel and the application layer

AN EXPLICIT HOST/DEVICE SPLIT, NOT A CLAIM THAT ALL WORK IS ON THE GPU

The shared C++/CUDA core implements three application jobs.

Job	Actual output
Sparse field stencil	New word values, the expression result and exact XOR delta.
Due-node proposal	One admitted next-cell proposal or stay / exit result per due node.
Calibrated remap	Destination ids for active binary samples under a supplied lookup table.

The host application assembles sparse fields, schedules deadlines, resolves target conflicts, compacts output, reconstructs paths, evaluates network/tour constraints and writes histories. This is a **hybrid application pipeline**. A persistent worker avoids creating a process per event, but each GPU job still uploads active input and downloads results.

Integer field definitions and transport

The kernel reads operator truth coefficients, direction masks and the $A = T$ record from the same packed atlas as the sparse data. There are no floating-point spatial calculations in these kernels. Addresses, latencies, indices and loop counters remain integer quantities with storage and execution cost.

Three access modes, no permanent cache pin

`texture` uses unsigned-element `tex1Dfetch`; `global` uses ordinary reads; `shared` stages the 64-byte current operator definition per block while reading sparse data through the texture path. Inputs and outputs are distinct, avoiding same-launch texture read-after-write assumptions. [N1]

The optional polar convention treats rows as log-radius labels and columns as angular cells. The reflected-seam option is an address topology, not quantum optics or proof of a physical Klein surface. Native-hop results are not automatically Euclidean distances.

The final source asks to leave zero space implicit. This implementation therefore allocates useful active state and work, rather than filling the remaining cache or VRAM with duplicate definitions just to report a full allocation. More occupancy is not itself a speed guarantee.

13 Sparse scaling and measured tradeoffs

AFTER-THE-FACT CPU COMPARISON / SAME ONE-STEP OUTPUT

A fixed pattern contains **2,054 active cells in 128 stored words**. Its candidate halo contains **157 word tasks**. The serialized sparse job is **2,868 bytes**, unchanged across the three finite domains below.

Declared cells	Sparse step	Dense packed step
65,536	12.25 μ s	4.56 μ s
4,194,304	11.20 μ s	260.13 μ s
1,073,741,824	11.30 μ s	148,364.68 μ s

The benchmark uses seven repetitions on an AMD EPYC 9V74 CPU, compiled with GCC and optimization. It compares the same unblocked rectangular Boolean first-arrival transition; all outputs agreed. Allocation, initial dense expansion and sparse task-list construction are outside the timed region. **These are not GPU results or whole-application timings.**

What the numbers mean

For the small domain, the direct dense sweep wins: sparse lookup has overhead. For a very large mostly empty domain, avoiding empty-space scanning makes the sparse step almost independent of the domain's unused area. That is the concrete benefit of the source's final sparse requirement.

A dense one-bit plane for the largest domain needs 128 MiB; the dense benchmark's three work planes require 384 MiB. One sparse field's 128 key/word pairs require 1,024 bytes, before container overhead. This is not a claim that a fully occupied billion-cell simulation also fits in that amount.

When not to generalize the measurement

A growing wave can make more words active. Crowds need node metadata and navigation information. Dense data can make sparse storage and searches worse than flat packed arrays. The kernel does not guarantee constant work when actual occupied support, connectivity or constraints grow.

Measured result: zeros outside the active neighborhood do not force a full scan.

Not measured: universal speed superiority, real-time human fidelity, energy savings, GPU cache hit rate or optimal use of the laptop's memory bandwidth.

14 Validation and remaining application boundaries

FINITE SOFTWARE EVIDENCE / SOURCE CLAIMS NOT PROMOTED TO RESULTS

Executed suite	Cases / assertions
All 3×3 allowed/seed assignments vs independent BFS	19,683
Literal source directional shifts	432
Boundary, morphology and wave profile comparisons	432
Arbitrary active-sample calibration maps	96
Delayed six-node crowd scenarios and replay	40
Network event graphs vs discrete-time oracle	120
Exact finite TSP graphs vs complete tour enumeration	72
Synthetic delivery-timestamp configurations	80
Four-input truth-table lane assertions	2,097,152
Bit-reversal checks	10,000
CPU memory/undefined-behavior sanitizer runs	PASS
CUDA / FPGA hardware execution	NOT RUN HERE

The suites compare independent representations or check specific invariants. They are not counts of human subjects, device calibrations or real deployments. The generated test data and exact source code are included.

Claims the document does not yet specify sufficiently

No supplied equation establishes an instant general TSP solver, a low-power FPGA implementation, an uncloneable biometric key, a neurochemical pleasure response, a physical IPD/ITD model or a consensus/currency mechanism. The generated responses in the attachment make these claims, but do not provide their proofs, measurements or complete protocols. [S31, pp. 4--6, 13--22, 29--31]

The simulation's shared rule and per-node Δt do not measure or rank human cultures, religions or biological groups. Any real calibration would need real input data and validation appropriate to that task; no population attribute is inferred by this software.

This is the scope of adherence: the documented binary field law, practical sparse/event direction and named software use cases have explicit implementations. Broader empirical, hardware and protocol properties are left visibly unestablished rather than substituted with a calculator example or declared solved.

15 Run the use cases and verify them

DELIVERABLES, COMMANDS AND SOURCE REFERENCES

The ZIP contains editable scenarios, CPU/CUDA source, the source-equation RTL block, binary emblem inputs, a formal source map, test fixtures, raw results and an event replay reader. It is a source distribution; no precompiled GPU executable is represented as tested.

CPU run without NVIDIA dependencies

```
cmake -S . -B build-cpu -DAXIS0_ENABLE_CUDA=OFF -DCMAKE_BUILD_TYPE=Release
cmake --build build-cpu --config Release --parallel
python run.py --kernel build-cpu/axis0_cpu --output results
```

On Windows use build-cpu/Release/axis0_cpu.exe. Run an editable crowd scenario rather than a fixed demonstration:

```
python app.py crowd examples/crowd.json --kernel build-cpu/axis0_cpu --out crowd-run
```

GPU build for the supplied laptop target

The build defaults to SM120 and requires CUDA 12.8 or later with a compatible host compiler. In a Visual Studio 2022 Developer PowerShell, run build_windows.ps1; Linux has build_linux.sh. Then validate:

```
python tests/applications.py --kernel build/axis0_cuda --backend texture --quick --json
verification/laptop_quick.json
```

The Windows GPU executable is build/Release/axis0_cuda.exe. After that test passes, run.py accepts the same backend. The CUDA APIs and target choice are grounded in the official documentation, not a claimed device run here. [N1--N2]

S31. Newly attached 31-page PDF beginning “ladybug anti violence tile in zaandam at the train station ...”. It is the requested basis; author prompts and generated responses are distinguished throughout. Full page-to-implementation coverage is in docs/SOURCE_MAP.md.

N1. [NVIDIA CUDA C++ Programming Guide 12.8.1](#): integer texture fetch, shared memory, texture coherence and resource constraints.

N2. [NVIDIA CUDA 12.8 Features Archive](#): SM120 compiler support. Official sources consulted 18 September 2026.